COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09-16-2026	**DATE SUBMITTED:**	09-16-2026
NAME:	Kherwin Emmanuel D. Milan	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 9.1 **Implementing Trees**

1. PROCEDURE OUTPUT

What procedures did you perform?

C/C++

```
#include <iostream>
#include "binaryTree.h"

int main(){

    //creating the root node
    BinaryTree<char>* root = new BinaryTree<char>('A');

    //creating children of root node "A"
    root->insertLeftChild('B');
    root->insertRightChild('C');

    //adding a child to a child
    root->getLeftChild()->insertLeftChild('D');
    root->getLeftChild()->insertRightChild('F');

    root->getRightChild()->insertLeftChild('E');

    //printing the root node
    std::cout << "Root value: " << root->getKey() << '\n';
    std::cout << "Left child of root: " << root->getLeftChild()->getKey() <<
'\n';
    std::cout << "Right child of root: " << root->getRightChild()->getKey() <<
'\n';

    std::cout << "Left child of root: " <<
root->getLeftChild()->getLeftChild()->getKey() << '\n';
    std::cout << "Right child of root: " <<
root->getLeftChild()->getRightChild()->getKey() << '\n';

    //replacing child F
    root->getLeftChild()->getRightChild()->changeKey('K');
```

```
    std::cout << "Right child of root: " <<
root->getLeftChild()->getRightChild()->getKey() << '\n';
    return 0;
}
```

C/C++

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

template<typename T>
class BinaryTree{
    private:
        T key;
        //pointers:
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        //constructor:
        BinaryTree(T rootObj);
        //destructor:
        ~BinaryTree();

        //adding/inserting a child node:
        void insertLeftChild(T newNode);
        void insertRightChild(T newNode);
        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();
        T getKey();
        void changeKey(T val);
        bool isLeaf();
};

template<typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}

template<typename T>
BinaryTree<T>::~BinaryTree(){
    delete leftChild;
    delete rightChild;
}
```

```cpp
template<typename T>
void BinaryTree<T>::insertLeftChild(T newNode){
    if (leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    }
    else {
        //CREATE A NEW NODE:
        BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
        //move the existing left child below the new Node
        newChild->leftChild = leftChild;
        leftChild = newChild;
    }
}

template<typename T>
void BinaryTree<T>::insertRightChild(T newNode){
    if (rightChild == nullptr){
        rightChild = new BinaryTree<T>(newNode);
    }
    else{
        //CREATE A NEW NODE:
        BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
        //move the existing right child below the new Node
        newChild->rightChild = rightChild;
        rightChild = newChild;
    }
}

template<typename T>
BinaryTree<T>* BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template<typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild(){
    return rightChild;
}


template<typename T>
T BinaryTree<T>::getKey(){
    return key;
}

template <typename T>
void BinaryTree<T>::changeKey(T val){
    key = val;
}
```

```cpp
template<typename T>
bool BinaryTree<T>::isLeaf(){
    return leftChild == nullptr && rightChild == nullptr;
}

#endif
```

```
Root value: A
Left child of root: B
Right child of root: C
Left child of root: D
Right child of root: F
Right child of root: K
```

Methods/functions of the BinaryTree:

1. BinaryTree(T rootObj)
   - This is the constructor. It creates a new node and sets its value as the root object. The left and right children are initially set to nullptr.
2. ~BinaryTree()
   - This is the destructor. It deletes the left and right child nodes when the tree is no longer needed.
3. insertLeftChild(T newNode)
   - This function adds a new node as the left child. If a left child already exists, the existing child is moved below the new node.
4. insertRightChild(T newNode)
   - This function adds a new node as the right child. If a right child already exists, the existing child is moved below the new node.
5. getLeftChild()
   - Returns the pointer to the left child of the current node.
6. getRightChild()
   - Returns the pointer to the right child of the current node.
7. getKey()
   - Returns the value or key stored in the current node.
8. changeKey(T val)
   - Changes the value stored in the current node to the given value.
9. isLeaf()
   - Checks if the node has no left or right child. It returns true if the node is a leaf and false otherwise.

Conclusion/Learnings:
In this activity, I learned how to create and manipulate a binary tree. I learned how to add left and right child nodes, access nodes, change a node's value, and check if a node is a leaf. The activity helped me understand how binary trees are organized using pointers.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.